



Documentacion Tecnica del Sistema

Gestion de Tickets TI y Control de Acceso RBAC

Documento: STD-2026-TI | Generado: 18/8/2026, 5:07:53 p. m.

1. Resumen del sistema

Sistema centralizado de mesa de ayuda para el departamento de Tecnologias de la Informacion. Gestiona el ciclo de vida completo de incidencias y requerimientos garantizando trazabilidad total sobre quien solicito, quien atendio y quien resolvió cada ticket, sobre una arquitectura de seguridad basada en control de acceso por roles con asignacion dinamica de permisos.

Componentes construidos

- API REST en Node.js con Express, autenticacion JWT y guard de permisos atomicos.
- Base de datos PostgreSQL con el esquema DDL de la especificacion mas bitacora y notificaciones.
- Canal de tiempo real con Socket.IO para notificaciones y actualizacion inmediata de tableros.
- Aplicacion web en React con Tailwind CSS e iconografia vectorial, sin uso de emojis.
- Aplicacion movil en React Native (Expo) que consume la misma API y el mismo canal de eventos.
- Motor documental PDF que emite actas, reportes y esta misma documentacion tecnica.
- Despliegue contenedorizado con Docker Compose para publicacion en servidor.

2. Arquitectura de la solucion

CAPA	TECNOLOGIA	RESPONSABILIDAD
Persistencia	PostgreSQL 16	Modelo relacional, integridad referencial y restricciones de dominio
Servicios	Node.js 20 + Express	API REST, reglas de negocio y control de acceso
Tiempo real	Socket.IO	Notificaciones y difusion de cambios por salas
Documental	PDFKit	Actas, reportes y documentacion tecnica en PDF
Cliente web	React 18 + Tailwind CSS	Interfaz administrativa y operativa
Cliente movil	React Native (Expo)	Atencion de tickets desde dispositivos moviles
Despliegue	Docker Compose + Nginx	Publicacion en servidor y proxy de API y sockets



El token JWT es la unica fuente de identidad: el identificador del usuario nunca se acepta desde el cliente en las operaciones de creacion, atencion o resolucion de tickets.



3. Modelo de datos

Estructura fisica implementada segun la especificacion, ampliada con las tablas de bitacora de auditoria y de notificaciones necesarias para la trazabilidad documental y el canal de tiempo real.

TABLA	PROPOSITO
areas	Catalogo de areas de la organizacion
categorias	Catalogo administrable de clasificacion de tickets
roles	Roles configurables desde el panel de administracion
permisos	Permisos atomicos del sistema agrupados por modulo
rol_permisos	Relacion N:M que materializa la matriz de permisos
usuarios	Cuentas con area, rol, estado y hash bcrypt de la contrasena
tickets	Requerimientos con trazabilidad de solicitante, asignado y resolutor
auditoria	Bitacora de cada accion ejecutada, base de los reportes PDF
notificaciones	Persistencia de los avisos emitidos por WebSockets
comentarios	Conversacion entre solicitante y tecnico sobre el ticket
adjuntos	Capturas de pantalla y documentos asociados al ticket



La categoria del ticket dejo de ser una lista fija en el codigo: ahora se valida contra el catalogo administrable de la tabla categorias, mantenible desde el panel de administracion. Al renombrar una categoría el cambio se propaga a los tickets ya clasificados.

Esquema DDL implementado

```
-- Sistema de Gestion de Tickets TI & Control de Acceso RBAC
-- Esquema fisico - PostgreSQL 16
-- Documento base: STD-2026-TI v1.0.0
--
-- 1. TABLA DE AREAS EMPRESARIALES
CREATE TABLE IF NOT EXISTS areas (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE,
    activo BOOLEAN DEFAULT TRUE,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
--
-- 2. TABLA DE ROLES (CONFIGURABLE DESDE EL PANEL DE ADMIN)
CREATE TABLE IF NOT EXISTS roles (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE,
    descripcion TEXT,
    activo BOOLEAN DEFAULT TRUE
);
--
-- 3. TABLA DE PERMISOS ATOMICOS DEL SISTEMA
CREATE TABLE IF NOT EXISTS permisos (
    id SERIAL PRIMARY KEY,
    codigo VARCHAR(100) NOT NULL UNIQUE,
    descripcion VARCHAR(255) NOT NULL,
    modulo VARCHAR(50) NOT NULL
);
--
-- 4. TABLA INTERMEDIA ROL-PERMISO (RELACION N:M)
```



```

CREATE TABLE IF NOT EXISTS rol_permisos (
    rol_id      INT REFERENCES roles(id) ON DELETE CASCADE,
    permiso_id INT REFERENCES permisos(id) ON DELETE CASCADE,
    PRIMARY KEY (rol_id, permiso_id)
);
-- 5. TABLA DE USUARIOS
CREATE TABLE IF NOT EXISTS usuarios (
    id          SERIAL PRIMARY KEY,
    nombre      VARCHAR(150) NOT NULL,
    usuario     VARCHAR(80) NOT NULL UNIQUE,
    email       VARCHAR(150) UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    area_id     INT NOT NULL REFERENCES areas(id),
    rol_id      INT NOT NULL REFERENCES roles(id),
    activo      BOOLEAN DEFAULT TRUE,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- 6. TABLA DE TICKETS
CREATE TABLE IF NOT EXISTS tickets (
    id          SERIAL PRIMARY KEY,
    titulo      VARCHAR(200) NOT NULL,
    descripcion TEXT NOT NULL,
    categoria   VARCHAR(50) NOT NULL,
    prioridad   VARCHAR(20) DEFAULT 'Media',
    estado      VARCHAR(20) DEFAULT 'Abierto',

    -- Trazabilidad de Roles / Usuarios en el Ticket
    solicitante_id INT NOT NULL REFERENCES usuarios(id),
    asignado_id    INT REFERENCES usuarios(id),
    resuelto_por_id INT REFERENCES usuarios(id),

    solucion_detalle TEXT,
    fecha_creacion   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    fecha_asignacion TIMESTAMP,
    fecha_resolucion TIMESTAMP
);
-- 7. BITACORA DE AUDITORIA (soporte a la documentacion PDF de cada accion)
CREATE TABLE IF NOT EXISTS auditoria (
    id          SERIAL PRIMARY KEY,
    usuario_id  INT REFERENCES usuarios(id),
    entidad     VARCHAR(50) NOT NULL,
    entidad_id  INT,
    accion      VARCHAR(60) NOT NULL,
    detalle     JSONB,
    ip          VARCHAR(64),
    fecha       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- 8. NOTIFICACIONES (persistencia de los eventos emitidos por sockets)
CREATE TABLE IF NOT EXISTS notificaciones (
    id          SERIAL PRIMARY KEY,
    usuario_id  INT NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,
    ticket_id   INT REFERENCES tickets(id) ON DELETE CASCADE,
    tipo        VARCHAR(50) NOT NULL,
    titulo      VARCHAR(150) NOT NULL,
    mensaje     TEXT NOT NULL,
    leida       BOOLEAN DEFAULT FALSE,
    fecha       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- INDICES DE APOYO
CREATE INDEX IF NOT EXISTS idx_tickets_estado ON tickets(estado);

```

```
CREATE INDEX IF NOT EXISTS idx_tickets_solicitante ON tickets(solicitante_id);Ø
CREATE INDEX IF NOT EXISTS idx_tickets_asignado ON tickets(asignado_id);Ø
CREATE INDEX IF NOT EXISTS idx_tickets_fecha ON tickets(fecha_creacion DESC);Ø
CREATE INDEX IF NOT EXISTS idx_usuarios_area ON usuarios(area_id);Ø
CREATE INDEX IF NOT EXISTS idx_auditoria_fecha ON auditoria(fecha DESC);Ø
CREATE INDEX IF NOT EXISTS idx_notif_usuario_leida ON notificaciones(usuario_id, leida);Ø
Ø
-- RESTRICCIONES DE DOMINIOØ
ALTER TABLE tickets DROP CONSTRAINT IF EXISTS chk_tickets_estado;Ø
ALTER TABLE tickets ADD CONSTRAINT chk_tickets_estadoØ
CHECK (estado IN ('Abierto','En Proceso','Resuelto','Cerrado'));Ø
Ø
ALTER TABLE tickets DROP CONSTRAINT IF EXISTS chk_tickets_prioridad;Ø
ALTER TABLE tickets ADD CONSTRAINT chk_tickets_prioridadØ
CHECK (prioridad IN ('Baja','Media','Alta','Critica'));
```

4. Diccionario de permisos atomicos

Permisos precargados en la base de datos. La interfaz de administracion construye los roles de forma dinamica a partir de este catalogo, agrupando las casillas de verificacion por modulo.

MODULO	CODIGO	DESCRIPCION DE LA ACCION
TICKETS	tickets.crear	Permite crear nuevas solicitudes de soporte.
TICKETS	tickets.ver_propios	Permite visualizar los tickets creados por el propio usuario.
TICKETS	tickets.ver_todos	Permite visualizar el listado completo de tickets del sistema.
TICKETS	tickets.responder	Permite asignarse un ticket y cambiar su estado a En Proceso.
TICKETS	tickets.resolver	Permite cerrar/resolver un ticket e ingresar la solucion tecnica.
ADMIN	admin.usuarios	Gestion CRUD de usuarios (crear, editar, activar/desactivar).
ADMIN	admin.roles	Gestion CRUD de roles y matriz de permisos.
ADMIN	admin.areas	Gestion del catalogo de areas de la empresa.
ADMIN	admin.categorias	Gestion del catalogo de categorias de tickets.
REPORTES	reportes.ver	Permite consultar el tablero de indicadores y reportes.
REPORTES	reportes.exportar	Permite exportar reportes y documentacion en formato PDF.

Roles precargados

ROL	PERMISOS CONCEDIDOS
admin	Todos los permisos del sistema
tecnico_l1	tickets.crear, ver_propios, ver_todos, responder, resolver, reportes.ver, reportes.exportar
tecnico_l2	Identico a tecnico_l1, previsto para escalamiento
cliente	tickets.crear, tickets.ver_propios

5. Control de acceso (middleware RBAC)

Cada endpoint valida el token JWT y luego evalua el guard de permisos. Los permisos vigentes del rol se resuelven contra la base de datos y se mantienen en una cache de corta duracion que se invalida al modificar la matriz de un rol, de modo que un cambio administrativo surte efecto de inmediato.

```
// Definicion de rutas protegidas
ticketsRouter.post('/', requierePermiso('tickets.crear'), crear);
ticketsRouter.put('/:id/tomar', requierePermiso('tickets.responder'), tomar);
ticketsRouter.put('/:id/resolver', requierePermiso('tickets.resolver'), resolver);
rolesRouter.post('/', requierePermiso('admin.roles'), crearRol);

// Guard: exige al menos uno de los codigos indicados
export const requierePermiso = (...codigos) => (req, _res, next) => {
  const posee = codigos.some((codigo) => req.usuario.permisos.includes(codigo));
  if (!posee) return next(HttpError.forbidden());
  return next();
};
```

→ 6. Ciclo de vida del ticket

ESTADO	DISPARADOR	EFFECTOS REGISTRADOS
Abierto	POST /tickets	solicitante_id = JWT.user_id; asignado_id y resuelto_por_id en NULL; fecha_creacion = NOW()
En Proceso	PUT /tickets/:id/tomar	asignado_id = JWT.user_id; fecha_asignacion = NOW(); notificacion al solicitante
En Proceso	PUT /tickets/:id/asignar	asignado_id = tecnico elegido; notificacion al tecnico asignado
Resuelto	PUT /tickets/:id/resolver	solucion_detalle registrada; resuelto_por_id = JWT.user_id; fecha_resolucion = NOW()
Cerrado	PUT /tickets/:id/cerrar	Cierre conforme por el solicitante o la mesa de ayuda



Cada transicion registra su accion en la bitacora de auditoria, emite el evento correspondiente por WebSockets y archiva automaticamente un acta PDF del ticket en el repositorio documental.

7. Endpoints expuestos

Prefijo base de la API: /api/v1

METODO	RUTA	PERMISO REQUERIDO	DESCRIPCION
POST	/auth/login	Publico	Autenticacion y emision del token JWT
GET	/auth/perfil	Autenticado	Perfil, rol y permisos vigentes
POST	/auth/cambiar-password	Autenticado	Cambio de contraseña propia
GET	/tickets	tickets.ver_propios / ver_todos	Listado con filtros y alcance por permiso
GET	/tickets/tablero	tickets.ver_propios / ver_todos	Indicadores y distribuciones
POST	/tickets	tickets.crear	Apertura de un ticket (estado Abierto)
GET	/tickets/:id	tickets.ver_propios / ver_todos	Detalle con bitacora de acciones
PUT	/tickets/:id/tomar	tickets.responder	Autoasignacion y paso a En Proceso
PUT	/tickets/:id/asignar	tickets.responder	Asignacion manual a otro tecnico
PUT	/tickets/:id/resolver	tickets.resolver	Registro de la solucion tecnica
PUT	/tickets/:id/cerrar	Solicitante o mesa de ayuda	Cierre conforme del ticket
GET	/tickets/:id/pdf	tickets.ver_propios / ver_todos	Acta PDF individual del ticket
GET	/tickets/reporte/pdf	reportes.exportar	Reporte consolidado en PDF
GET	/tickets/:id/comentarios	Participante del ticket	Conversacion del ticket
POST	/tickets/:id/comentarios	Participante del ticket	Mensaje con hasta cinco adjuntos
GET	/adjuntos/:id	Participante del ticket	Descarga del archivo adjunto
GET	/areas	Autenticado	Catalogo de areas
POST	/areas	admin.areas	Alta de area
PUT	/areas/:id	admin.areas	Edicion de area
DELETE	/areas/:id	admin.areas	Baja logica de area
GET	/roles	admin.roles / admin.usuarios	Roles con su matriz de permisos
POST	/roles	admin.roles	Creacion de rol con permisos



METODO	RUTA	PERMISO REQUERIDO	DESCRIPCION
PUT	/roles/:id	admin.roles	Reemplazo integral de la matriz del rol
DELETE	/roles/:id	admin.roles	Eliminacion de rol sin usuarios
GET	/permisos	admin.roles	Catalogo de permisos agrupado por
GET	/categorias	Autenticado	Catalogo de categorias de ticket
POST	/categorias	admin.categorias	Alta de categoria con color e icono
PUT	/categorias/:id	admin.categorias	Edicion con propagacion del nombre a los tickets
DELETE	/categorias/:id	admin.categorias	Baja logica de categoria
GET	/usuarios	admin.usuarios	Listado con filtros
GET	/usuarios/tecnicos	tickets.ver_todos	Tecnicos disponibles para asignacion
POST	/usuarios	admin.usuarios	Alta de usuario con hash bcrypt
PUT	/usuarios/:id	admin.usuarios	Edicion de usuario
DELETE	/usuarios/:id	admin.usuarios	Baja logica de usuario
GET	/notificaciones	Autenticado	Bandeja de notificaciones propias
PUT	/notificaciones/:id/leida	Autenticado	Marcado individual como leida
GET	/auditoria	reportes.ver / admin.usuarios	Bitacora de acciones
GET	/auditoria/pdf	reportes.exportar	Bitacora de auditoria en PDF
GET	/auditoria/matriz-rbac/pdf	admin.roles	Matriz de roles y permisos en PDF

 8. Canal de tiempo real

El socket se autentica con el mismo token JWT en el handshake. Cada usuario se une a una sala personal y los perfiles con permiso tickets.ver_todos se incorporan ademas a la sala del equipo tecnico.

EVENTO	DIRECCION	DESCRIPCION
conexion:establecida	Servidor a cliente	Confirma la autentificacion del socket y las salas asignadas
ticket:suscribir	Cliente a servidor	Suscribe al canal de un ticket especifico
ticket:desuscribir	Cliente a servidor	Abandona el canal de un ticket
ticket:creado	Servidor a cliente	Nuevo ticket registrado en la mesa de ayuda
ticket:actualizado	Servidor a cliente	Cambio de estado, asignacion o cierre
ticket:resuelto	Servidor a cliente	Registro de la solucion tecnica
notificacion:nueva	Servidor a cliente	Notificacion personal para el destinatario
comentario:nuevo	Servidor a cliente	Mensaje nuevo en la conversacion del ticket

 9. Modulo de documentacion en PDF

Todas las salidas documentales del sistema se generan con un unico motor que garantiza identidad visual uniforme: encabezado institucional, iconografia vectorial, tablas con encabezado repetido y pie de pagina con la autoria del modulo en cada hoja. No se emplean emojis en ningun documento.

DOCUMENTO	ORIGEN	MOMENTO DE EMISION
Acta de ticket	GET /tickets/:id/pdf	Bajo demanda y automatica en cada transicion
Reporte de gestion	GET /tickets/reporte/pdf	Bajo demanda con los filtros vigentes
Bitacora de auditoria	GET /auditoria/pdf	Bajo demanda por periodo y entidad
Matriz de roles y permisos	GET /auditoria/matriz-rbac/pdf	Bajo demanda desde el panel de roles
Documentacion tecnica	node docs/generator/generate-docs.js	En cada actualizacion del sistema

Iconografia disponible en los documentos

- ☒ ticket, usuario, escudo y engranaje para identidad de modulos y secciones.
- ☒ reloj, check y alerta para estados, cumplimiento y prioridades criticas.
- ☒ documento, grafico, baseDatos, flujo y red para reportes y arquitectura.

10. Despliegue en servidor

La solución se publica con Docker Compose. La base de datos aplica automáticamente el esquema y la carga inicial en su primer arranque; la aplicación web se sirve mediante Nginx, que además actúa como proxy de la API y del canal de WebSockets.

```
cp .env.example .env          # definir credenciales y JWT_SECRET
docker compose build
docker compose up -d

# Servicios publicados
web  -> http://localhost:8080
api  -> http://localhost:4000/api/v1
salud -> http://localhost:4000/salud
```

 Antes de publicar en producción es obligatorio reemplazar JWT_SECRET y la contraseña de la base de datos, y cambiar la clave del usuario administrador inicial.

Ejecución en entorno de desarrollo

```
# 1. Base de datos (PostgreSQL local)
createdb tickets_ti
cd backend && npm install && npm run migrate

# 2. API
npm run dev

# 3. Aplicación web
cd ../frontend && npm install && npm run dev

# 4. Aplicación móvil
cd ../mobile && npm install && npm start
```

11. Inventario de componentes

COMPONENTE	RUTA	CONTENIDO
Base de datos	db/01_schema.sql	Esquema DDL completo e índices
Base de datos	db/02_seed.sql	Áreas, roles, permisos y usuario administrador
Base de datos	db/03_categorias.sql	Catálogo administrable de categorías
Base de datos	db/04_comentarios.sql	Conversación y adjuntos del ticket
API	backend/src/app.js	Composición de middlewares y montaje de rutas
API	backend/src/server.js	Servidor HTTP, sockets y apagado ordenado
API	backend/src/middleware/auth.js	Verificación JWT para HTTP y para sockets
API	backend/src/middleware/rbac.js	Guard RequierePermiso sobre permisos atómicos
API	backend/src/modules/tickets/	Ciclo de vida completo del ticket
API	backend/src/realtime/socket.js	Canal de tiempo real y gestión de salas
API	backend/src/services/pdf/	Motor documental: iconos, plantilla y reportes
Web	frontend/src/context/	Sesión, permisos y notificaciones en vivo
Web	frontend/src/pages/	Tablero, tickets, administración y auditoría
Móvil	mobile/src/screens/	Pantallas Expo con el mismo modelo de permisos



COMPONENTE	RUTA	CONTENIDO
Despliegue	docker-compose.yml	Orquestacion de base de datos, API y web

 12. Acceso inicial y control de versiones

USUARIO ADMINISTRADOR admin	CONTRASENA INICIAL Admin123*
AREA ASIGNADA Tecnologias de la Informacion	ROL ASIGNADO admin

VERSION	FECHA	DESCRIPCION	RESPONSABLE
1.0.0	18/8/2026	Construccion inicial completa del sistema segun STD-2026-TI	Ing. Edgar Rojas Apaza



